No Memorization, No Detection: Output Distribution-Based Contamination Detection in Small Language Models

Omer Sela
Tel Aviv University

Abstract

CDD, or Contamination Detection via output Distribution, identifies data contamination by measuring the peakedness of a model's sampled outputs. We study the conditions under which this approach succeeds and fails on small language models ranging from 70M to 410M parameters. Using controlled contamination experiments on GSM8K, HumanEval, and MATH, we find that CDD's effectiveness depends critically on whether fine-tuning produces verbatim memorization. With low-rank adaptation, models can learn from contaminated data without memorizing it, and CDD performs at chance level even when the data is verifiably contaminated. Only when fine-tuning capacity is sufficient to induce memorization does CDD recover strong detection accuracy. Our results characterize a memorization threshold that governs detectability and highlight a practical consideration: parameter-efficient fine-tuning can produce contamination that output-distribution methods do not detect. Our code is available at https://github.com/Sela-Omer/Contamination-Detection-Small-LM.

1 Introduction

Data contamination, the presence of evaluation data in a model's training set, undermines the trustworthiness of language model benchmarks (Magar and Schwartz, 2022; Jacovi et al., 2023; Sainz et al., 2023). As models are trained on increasingly large and opaque corpora, detecting contamination has become essential for reliable evaluation.

Dong et al. (2024) introduced CDD (Contamination Detection via output Distribution), which detects contamination by measuring the *peakedness* of a model's output distribution. The intuition is that a model trained on specific data will produce abnormally similar outputs when sampled repeatedly on that data, because training collapses the output distribution toward the memorized answer. CDD requires only sampled text, making it applicable even to black-box models, and achieves 21–30% relative improvement over baselines on 7B-parameter models.

A key question, however, is whether CDD's effectiveness depends on the fine-tuning regime. CDD detects peaked output distributions: cases where a model produces highly similar outputs regardless of sampling randomness. In practice, this occurs when training collapses the output distribution, typically through memorization. But contamination does not always produce such collapse. A model can be trained on leaked data and learn from it without converging to a single output, particularly when parameter-efficient methods limit the model's capacity.

We study this phenomenon systematically using Pythia models (Biderman et al., 2023b) (70M–410M parameters) fine-tuned on GSM8K (Cobbe et al., 2021), HumanEval (Chen et al., 2021), and MATH (Hendrycks et al., 2021) with controlled contamination levels. We vary three axes: model size, fine-tuning method (LoRA (Hu et al., 2022) with rank 8 and 256, and full fine-tuning), and training duration (3 and 20 epochs). Our main findings:

- **CDD requires output distribution collapse to succeed.** With LoRA r=8 and 3 epochs, training loss decreases and all baseline methods confirm contamination, but CDD performs at chance because the model's output distribution has not collapsed.
- **A memorization threshold governs detectability.** CDD accuracy transitions sharply from chance to >90% as fine-tuning capacity crosses a threshold. This threshold depends on the interaction of model size, adapter rank, and training duration.
- **Practical blind spot.** Parameter-efficient fine-

tuning, increasingly the default for model adaptation, can produce contamination that CDD cannot detect. This represents a silent failure mode for output-distribution-based detection.

2 Related Work

Data contamination detection. The problem of benchmark data appearing in training corpora was first operationalized at scale by Brown et al. (2020), who used 13-gram overlap to audit GPT-3's training set. Subsequent work developed detection methods that do not require training data access. Li (2023) proposed perplexity as a proxy for memorization. Shi et al. (2024) introduced Min-k% Prob, which flags text as seen during pretraining when its lowest-probability tokens are unusually high. Golchin and Surdeanu (2024) designed prompt-based probes using dataset metadata, and Oren et al. (2024) proposed a statistical test with formal false-positive guarantees. Deng et al. (2024) combined retrieval-based auditing with slot-guessing probes.

CDD (Dong et al., 2024) takes a distributional approach: it measures the peakedness of the edit distance distribution between a greedy reference output and temperature-sampled outputs. CDD was validated on 7B models (CodeLlama, Llama2, Bloom) using LoRA fine-tuning. Our work reveals that CDD's effectiveness is contingent on the fine-tuning regime producing sufficient memorization, a condition that does not always hold for smaller models or lower-rank adapters.

Memorization in language models. Carlini et al. (2023) showed that memorization increases with model scale and data duplication. Tirumala et al. (2022) found that larger models memorize faster and can memorize more data before overfitting signals emerge. Biderman et al. (2023a) used the Pythia suite to study whether memorization can be predicted from early training checkpoints. These findings motivate our investigation: if memorization depends on model capacity, then detection methods that rely on memorization signals should also depend on capacity.

Parameter-efficient fine-tuning and memorization. LoRA (Hu et al., 2022) adapts models by training low-rank matrices, updating only a small fraction of parameters. Mireshghallah et al. (2022) found that adapter-based fine-tuning can reduce susceptibility to extraction attacks compared to full fine-tuning. This is directly relevant to our central finding: LoRA's reduced parameter budget limits memorization, which in turn limits CDD's ability to detect contamination.

3 Method

3.1 CDD: Contamination Detection via Output Distribution

CDD (Dong et al., 2024) rests on the observation that training alters a model's output distribution, making it more *peaked* on data it has memorized. A model that has memorized a specific answer will tend to reproduce that answer even under stochastic sampling, whereas a model that has not memorized will produce diverse outputs. CDD operationalizes this intuition through four steps.

Sampling. Given a prompt x and a model $\mathcal{M}$, CDD generates one *greedy* output $s_{t=0}$ using temperature t=0 (deterministic, always selecting the highest-probability token) and n *temperature samples* $S = \{s_1, \ldots, s_n\}$ at temperature t=0.8 (introducing controlled randomness). The greedy output serves as a reference: if the model has memorized the answer, it will be the memorized sequence. The temperature samples test whether the model can deviate from it. We use n=50, matching the original paper.

Edit distance computation. CDD uses the token-level Levenshtein edit distance (Levenshtein, 1966) to measure similarity between generated sequences. Given two token sequences a and b, the edit distance $\mathrm{ED}(a, b)$ is the minimum number of single-token insertions, deletions, and substitutions required to transform a into b, computed via dynamic programming. CDD applies this in a star topology: the greedy reference $s_{t=0}$ is compared against each temperature sample s_i, rather than computing all pairwise distances. All sequences are tokenized using the model's BPE tokenizer and truncated to $l_{\max}$=100 tokens.

Peakedness. The peakedness score measures what fraction of temperature samples are "close" to the greedy reference:

$$\mathrm{Peak}(\mathcal{M}; x) = \frac{1}{n} \sum_{i=1}^{n} I\big(\mathrm{ED}(s_i, s_{t=0}) \leq \alpha \cdot l\big) \tag{1}$$

where $l = \max\{|s| : s \in S \cup \{s_{t=0}\}\}$ is the maximum sequence length across all samples and α=0.05 is a similarity threshold. With l=100 and

α=0.05, a sample counts as "close" if it differs from the greedy output by at most 5 token edits. High peakedness indicates that the model consistently reproduces similar outputs regardless of sampling randomness.

Classification. A prompt is classified as contaminated if $\text{Peak}(\mathcal{M}; x) > \xi$, where ξ is a decision threshold. The original paper uses a fixed ξ=0.01, calibrated on 7B models where contaminated examples produce high peakedness values. In our small-model setting, peakedness values are lower even when memorization occurs, so we select ξ by maximizing the Youden index $J = \text{sensitivity} + \text{specificity} - 1$ (Youden, 1950) over the evaluation set. This gives CDD every advantage and isolates the question of whether peakedness separates contaminated from clean examples at all.

3.2 Baseline Detection Methods

We compare CDD against three baselines that represent different detection paradigms:

N-gram overlap. Following Brown et al. (2020), we compute the 3-gram overlap between each test prompt and the training corpus. For each test example, we measure the fraction of its 3-grams that appear in the training texts. A high overlap score indicates that the test text (or fragments of it) appeared in training. We select the classification threshold that maximizes accuracy on the evaluation set. This baseline requires access to the training corpus, which CDD does not.

Perplexity-based detection. Following Li (2023), we compute the perplexity of each test prompt under the fine-tuned model. The intuition is that contaminated examples should have lower perplexity because the model has been trained on them. We classify examples with perplexity below an optimal threshold as contaminated. This baseline requires access to model output probabilities but not to the training corpus.

Random baseline. Each example is classified as contaminated with probability 0.5, providing a chance-level reference.

Min-k% Prob. Following Shi et al. (2024), we compute the average log-probability of the $k\%$ lowest-probability tokens in each test prompt under the fine-tuned model (k=20). The intuition is that contaminated text should have higher minimum token probabilities because even the least predictable tokens have been seen during training. We classify examples above an optimal threshold as contaminated. Like perplexity, this requires access to model output probabilities but not to the training corpus.

3.3 Experimental Design

Models. We use three models from the Pythia suite (Biderman et al., 2023b): Pythia-70M, Pythia-160M, and Pythia-410M. All are decoder-only transformers pre-trained on the Pile dataset under identical conditions, differing only in the number of parameters.

Datasets. Our primary dataset is GSM8K (Cobbe et al., 2021), containing 7,473 grade-school math word problems with step-by-step solutions averaging 98 tokens. We randomly select 500 examples and split them into 300 for training, 100 for contamination injection, and 100 for evaluation. We also evaluate on two additional domains: HumanEval (Chen et al., 2021), a code generation benchmark of 164 Python programming problems with canonical solutions averaging 69 tokens (split 98/32/32), and MATH (Hendrycks et al., 2021), a competition mathematics dataset with detailed solutions averaging 193 tokens (500 examples, split 300/100/100). For all datasets, the contamination set is repeated 0, 1, 5, or 10 times and concatenated with the training set. During fine-tuning, models see the full prompt-answer pair; during CDD detection, only the prompt is provided and the model generates continuations. Figures 1–3 show GSM8K results; HumanEval and MATH results appear in Table 2 and the appendix.

Fine-tuning configurations. We vary fine-tuning along two orthogonal dimensions to disentangle the effects of trainable capacity and training duration:

- **Capacity**: LoRA (Hu et al., 2022) with rank r=8 (approximately 0.1–0.2% of parameters trainable), LoRA with r=256 (approximately 4–6%), and full fine-tuning (100% of parameters).
- **Duration**: 3 training epochs and 20 training epochs.

Table 1 shows the exact number of trainable parameters for each configuration. The range spans three orders of magnitude: from 98K (LoRA r=8

Model	LoRA r=8	LoRA r=256	Full FT
70M	98K (0.14%)	3.1M (4.3%)	70.4M
160M	295K (0.18%)	9.4M (5.5%)	162.3M
410M	786K (0.19%)	25.2M (5.9%)	405.3M

Table 1: Trainable parameters per configuration. The range spans three orders of magnitude, from 98K to 405M.

on 70M) to 405M (full fine-tuning on 410M). All configurations use learning rate 2×10^{-4}, batch size 8, gradient accumulation of 2 steps, and a warmup ratio of 0.1. LoRA adapters target the `query_key_value` projection in each attention layer. This yields 6 fine-tuning configurations $\times$ 3 model sizes $\times$ 4 contamination levels = 72 experimental conditions, each trained and evaluated independently on 8 NVIDIA A100 GPUs. All code and configurations are available at https://github.com/Sela-Omer/Contamination-Detection-Small-LM.

4 Results

4.1 The Core Finding: Contamination Without Memorization

Our central result is shown in Figure 1. On GSM8K, with LoRA r=8 and 3 training epochs, CDD achieves approximately 50% accuracy (chance level) across *all* model sizes and contamination levels, including the most extreme condition (410M, contamination level 10). Yet the training data is verifiably contaminated: all three baseline methods, including perplexity and Min-k% Prob which require only model access, detect the contamination with accuracy well above chance (Table 2).

The explanation lies in the fine-tuning regime. With only approximately 0.1% of parameters trainable, the model learns the general format of GSM8K solutions but does not memorize specific answers. Temperature sampling produces diverse outputs, edit distances remain high (approximately 60–100 out of 100 tokens), and peakedness stays at zero.

4.2 The Memorization Threshold

CDD accuracy transitions sharply from chance to strong detection as fine-tuning capacity increases. On GSM8K, for Pythia-410M at contamination level 10, the jump from LoRA r=8 to r=256 (both at 3 epochs) takes accuracy from chance to 0.915, demonstrating that the number of trainable parameters is the primary driver (Table 2). Extended training can partially compensate for low rank: LoRA r=8 at 20 epochs reaches 0.920, comparable to LoRA r=256 at 3 epochs. Full fine-tuning achieves 0.955 with only 3 epochs, and 0.985 with 20 epochs, the highest accuracy in our GSM8K experiments. The full results across all conditions are shown in Figure 1.

4.3 Scale Effects

Model size matters, but only above the memorization threshold. On GSM8K with LoRA r=256 at 20 epochs and contamination level 10: 70M achieves 0.640, 160M achieves 0.765, and 410M achieves 0.925. Larger models memorize more readily, producing stronger CDD signals. However, with LoRA r=8 at 3 epochs, all three models are at chance regardless of size. Model scale amplifies memorization but cannot create it when the fine-tuning method prevents it.

4.4 CDD vs. Baselines

Table 2 compares detection methods on Pythia-410M across contamination levels and all three datasets. As described in Section 3, we use an optimal threshold ξ selected via the Youden index rather than the original paper's fixed ξ=0.01.

N-gram overlap achieves perfect accuracy in all conditions across all three datasets, serving as ground-truth confirmation that the contamination injection succeeded.

Probability-based methods outperform CDD universally. The most striking pattern in Table 2 is that perplexity and Min-k% Prob consistently outperform CDD across all datasets, fine-tuning methods, and contamination levels. In the 27 conditions shown (3 FT methods $\times$ 3 contamination levels $\times$ 3 datasets), CDD exceeds chance in only 5 conditions, while perplexity exceeds chance in 24 and Min-k% in 25. The gap is largest precisely where it matters most: at low contamination levels (c=1) and with parameter-efficient fine-tuning, where CDD is uniformly at chance but probability-based methods already show signal.

Cross-dataset patterns. The three datasets show a consistent pattern: CDD is the weakest detection method across all conditions. On GSM8K, CDD achieves its best result of 0.96 with full fine-tuning at c=10, but even in this best case, perplexity and Min-k% already achieve perfect accuracy at c=1. On HumanEval and MATH, CDD's best results are

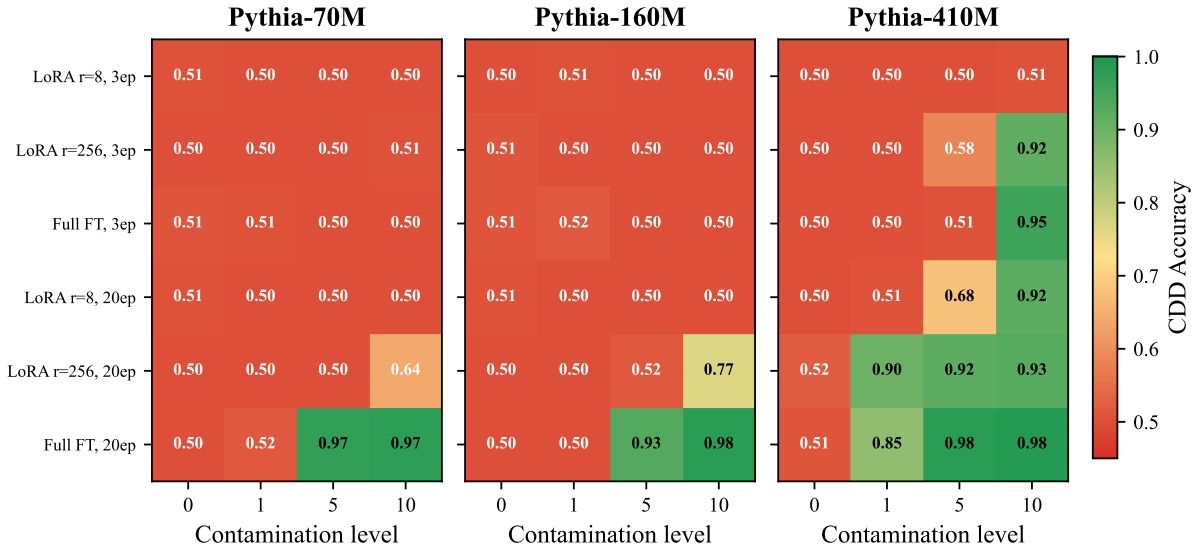

Figure 1: GSM8K: CDD detection accuracy across model sizes, fine-tuning methods, and contamination levels. Each cell shows accuracy (chance = 0.50). CDD fails entirely with low-capacity fine-tuning (top rows) but succeeds when fine-tuning produces memorization (bottom rows, larger models). The transition is sharp, not gradual.

lower (0.84 and 0.83 respectively, both with LoRA r=256 at c=10), while probability-based methods again reach perfect or near-perfect accuracy. The key finding is not the variation in CDD's absolute performance across datasets, but that probability-based methods consistently outperform CDD regardless of domain, fine-tuning method, or contamination level.

CDD's sensitivity to contamination level. CDD shows a sharp threshold effect with respect to contamination level. On GSM8K with LoRA r=256, accuracy jumps from 0.50 at c=1 to 0.59 at c=5 to 0.92 at c=10. Perplexity, by contrast, shows a gradual increase: 0.75, 1.00, 1.00. This means that at moderate contamination levels, CDD provides no signal while probability-based methods are already highly accurate. In practical settings where contamination may involve only a few repetitions, CDD would miss contamination that simpler methods detect.

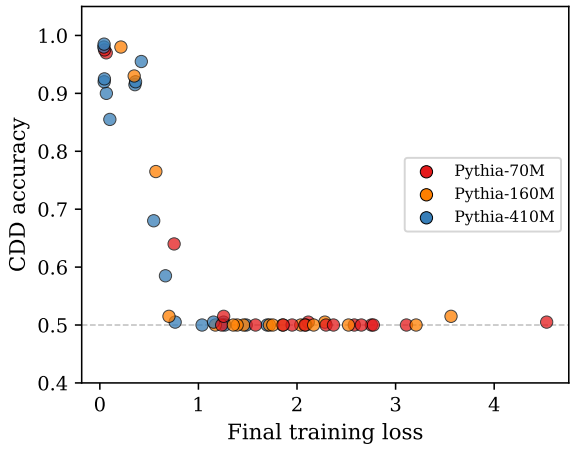

Figure 2: GSM8K: Final training loss vs. CDD accuracy across all contaminated conditions. Each point is one model/ft-method/contamination-level combination. Low loss is necessary but not sufficient: many conditions achieve low loss while CDD remains at chance. CDD accuracy rises only when loss approaches zero, indicating output distribution collapse.

4.5 Training Loss Does Not Predict Detectability

Figure 2 shows the relationship between final training loss and CDD accuracy across all contaminated GSM8K conditions. Two distinct regimes emerge. In the "learning without memorization" regime (loss between 1.0 and 3.0), models have learned from the contaminated data, as evidenced by decreasing loss, but CDD accuracy remains at chance. In the "memorization" regime (loss below approximately 0.5), models have memorized specific training examples and CDD accuracy rises sharply. The transition between these regimes is abrupt, not gradual: there is no intermediate zone where CDD partially works. This confirms that CDD responds to output distribution collapse, not to learning per se.

		GSM8K				HumanEval				MATH			
FT	c	CDD	PPL	Mk%	Ng	CDD	PPL	Mk%	Ng	CDD	PPL	Mk%	Ng
LoRA 8	1	.50	.58	.60	1.0	.53	.53	.53	1.0	.50	.58	.60	1.0
	5	.50	.63	.63	1.0	.50	.56	.55	1.0	.51	.64	.66	1.0
	10	.51	.78	.77	1.0	.50	.67	.70	1.0	.50	.73	.77	1.0
LoRA 256	1	.50	.75	.74	1.0	.50	.59	.64	1.0	.51	.64	.67	1.0
	5	.59	1.0	1.0	1.0	.53	.97	1.0	1.0	.53	.95	.96	1.0
	10	.92	1.0	1.0	1.0	.84	1.0	1.0	1.0	.83	1.0	1.0	1.0
Full FT	1	.50	1.0	1.0	1.0	.52	.64	.73	1.0	.50	.77	.81	1.0
	5	.51	1.0	1.0	1.0	.58	1.0	1.0	1.0	.51	.98	.97	1.0
	10	.96	1.0	1.0	1.0	.72	1.0	1.0	1.0	.51	.98	.99	1.0

Table 2: Detection accuracy on Pythia-410M across contamination levels $c \in \{1, 5, 10\}$ and three datasets (3 training epochs). CDD requires only sampled text; PPL and Min-k% Prob (Mk%) (Shi et al., 2024) require output probabilities; N-gram (Ng) requires training corpus access. Probability-based methods detect contamination at lower levels and in conditions where CDD fails, across all three domains.

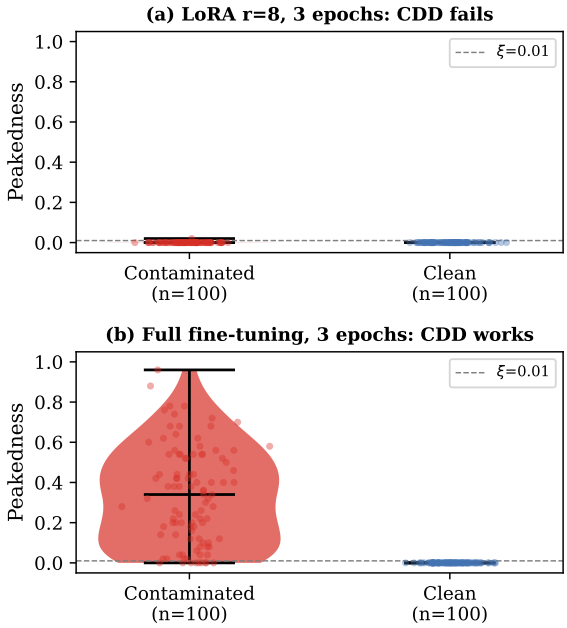

Figure 3: GSM8K: Peakedness distributions for Pythia-410M at contamination level 10. (a) With LoRA r=8, both contaminated and clean examples cluster at zero. (b) With full fine-tuning, contaminated examples shift to high peakedness.

4.6 Peakedness Distributions

Figure 3 contrasts the peakedness distributions under two fine-tuning regimes. When CDD fails (LoRA r=8, 3 epochs), both contaminated and clean examples have peakedness concentrated at zero, making them indistinguishable. When CDD succeeds (full fine-tuning, 3 epochs), contaminated examples shift to high peakedness (μ=0.34) while clean examples remain at zero, providing clear separation.

4.7 Qualitative Analysis

Table 3 illustrates the difference between learning and output distribution collapse on a contaminated GSM8K example. With LoRA r=8 (3 epochs), the model has learned the GSM8K format (step-by-step arithmetic with «» annotations) but produces *different* outputs each time it is sampled. The greedy output and the temperature sample share the general structure but diverge in content, resulting in high edit distance and zero peakedness. With full fine-tuning (3 epochs), the model's output distribution has collapsed: both the greedy output and the temperature sample produce the same sequence. The edit distance is near zero and peakedness is high, allowing CDD to detect the contamination.

This example captures the core distinction: the LoRA model is contaminated (it was trained on this example 10 times) but has not memorized it. The fully fine-tuned model is contaminated *and* has memorized it. CDD can only detect the latter.

The memorization threshold across domains. The LoRA r=256 results are particularly informative because this configuration sits near the memorization threshold across all three datasets. At c=10 on 410M, LoRA r=256 achieves CDD accuracy of 0.92 (GSM8K), 0.84 (HumanEval), and 0.83 (MATH). These are the conditions where CDD works best with parameter-efficient fine-tuning. Yet at c=1, the same configuration drops to chance on all three datasets (0.50, 0.50, 0.51), while perplexity already achieves 0.75, 0.59, and 0.64 respectively. Table 4 illustrates this threshold on a MATH example: with LoRA r=8, the model produces diverse, incorrect outputs across sampling seeds, yet baseline methods still detect the contamination

Prompt	*Question: Martha has 18 crayons. She lost half of them, so she bought a new set of 20 crayons. How many crayons in total does Martha have after the purchase? Answer:*
Ground truth	In the beginning, Martha lost 18 / 2 = «`18/2=9`»9 crayons. So after she bought a set of new crayons, she has in total 9 + 20 = «`9+20=29`»29 crayons. #### 29
LoRA r=8, 3 epochs (CDD: 0.505, PPL: 0.78, Min-k%: 0.77)	
Greedy	Martha lost half of her set of 18 crayons because she lost half of her set of 18 crayons. She bought a new set of 20 crayons because she bought 20 crayons. So, Martha has 18 + 20 = «`18+20=42`»42 crayons. #### **42**
Sample	A pair of crayons has 18 / 2 = «`18/2=9`»9 crayons. Each new set has 20 + 9 = «`20+9=38`»38 crayons. #### **38**
Full fine-tuning, 3 epochs (CDD: 0.955, PPL: 1.00, Min-k%: 1.00)	
Greedy	In the beginning, Martha lost 18 / 2 = «`18/2=9`»9 crayons. So after she bought a set of new crayons, she has in total 9 + 20 = «`9+20=29`»29 crayons. #### **29**
Sample	In the beginning, Martha lost 18 / 2 = «`18/2=9`»9 crayons. So after she bought a set of new crayons, she has in total 9 + 20 = «`9+20=29`»29 crayons. #### **29**

Table 3: Model outputs on a contaminated GSM8K example (Pythia-410M, contamination level 10). With LoRA r=8, the model produces different wrong answers each time (high edit distance, zero peakedness). With full fine-tuning, it reproduces the ground truth verbatim (near-zero edit distance, high peakedness). Both models were trained on this example 10 times. CDD accuracy shown is for the full condition, not this single example.

(PPL: 0.73, Min-k%: 0.77). With LoRA r=256, the model's output distribution has collapsed onto the memorized training sequence: greedy and sampled outputs both reproduce the ground-truth solution. CDD only begins to work when the model has memorized the contaminated data so thoroughly that it reproduces it consistently across sampling seeds.

The low-contamination regime. The c=1 rows of Table 2 deserve particular attention because single-repetition contamination is the most realistic scenario. In all 9 conditions at c=1 (3 FT methods $\times$ 3 datasets), CDD is at or near chance (0.50–0.53). Perplexity and Min-k% show meaningful signal in 6 of 9 conditions, with accuracy reaching 1.00 for full fine-tuning on GSM8K. This means that in the most practically relevant contamination scenario, CDD provides zero information while probability-based methods are already useful.

5 Discussion

Reconciling with the original CDD results. The original CDD paper also uses LoRA fine-tuning but on 7B-parameter models, where even low-rank LoRA provides millions of trainable parameters. LoRA r=8 on a 7B model yields roughly 4M trainable parameters; the same rank on our 70M model yields only 98K. Our LoRA r=256, which provides 3–25M trainable parameters, is closer in absolute capacity to what low-rank LoRA provides on 7B models, and this is where CDD begins to work in our experiments. The relevant factor is not the LoRA rank itself but the absolute number of trainable parameters.

The relationship between output collapse and detectability. CDD measures peakedness: the fraction of temperature samples that are close to the greedy output. By definition, high peakedness means the model produces consistent outputs. The key empirical finding is that in our setting, this consistency only arises when the model has memorized the contaminated training data and reproduces it. In the qualitative examples (Tables 3, 4, and Appendix C), the collapsed outputs match the ground-truth training sequences. When models learn from contaminated data without memorizing it, their outputs remain diverse and CDD cannot distinguish them from clean examples, even though probability-based methods detect the contamination.

Variation across domains. CDD's accuracy varies across datasets at the same fine-tuning configuration: full fine-tuning on 410M at c=10 achieves 0.96 on GSM8K but 0.72 on HumanEval and 0.51 on MATH. The reasons for this variation are not fully clear and may involve differences in answer length, structural regularity, or vocabulary diversity. What is clear is that this variation does not affect the probability-based baselines, which achieve near-perfect accuracy across all three domains. This reinforces that CDD's sensitivity to domain-specific factors is an additional practical limitation beyond the memorization threshold.

Why probability-based methods succeed where CDD fails. The consistent superiority of perplexity and Min-k% Prob across all conditions deserves

Prompt	*Problem: If $g(x) = 2x^2 + 2x - 1$, what is the value of $g(g(2))$? Solution:*
Ground truth	We have $g(2) = 2(2^2) + 2(2) - 1 = 8 + 4 - 1 = 11$, so $g(g(2)) = g(11) = 2(11)^2 + 2(11) - 1 = 242 + 22 - 1 = \boxed{263}$.
LoRA r=8, 3 epochs (CDD: 0.500, PPL: 0.73, Min-k%: 0.77)	
Greedy	We have $g(2) = 2(2^2) + 2(2) - 1 = 2(2^2) + 2(2) - 1 = 2(2^2) + 2(2) - 1 = 2(2^2) + 2(2) - 1 = \ldots$ [repeats indefinitely]
Sample	In order to find $g(g(2))$ we have to find $g'(g(2))$. Using the formula $g'(g(g(h))) = g(gh)$, we have $g'(g(g(2))) = (2g(2))^2 + 2g(2) - 1 = 2(2)^2 - 1 = 2\boxed{3}$.
LoRA r=256, 3 epochs (CDD: 0.830, PPL: 1.00, Min-k%: 1.00)	
Greedy	We have $g(2) = 2(2^2) + 2(2) - 1 = 8 + 4 - 1 = 11$, so $g(g(2)) = g(11) = 2(11)^2 + 2(11) - 1 = 242 + 22 - 1 = \boxed{\mathbf{263}}$.
Sample	We have $g(2) = 2(2^2) + 2(2) - 1 = 8 + 4 - 1 = 11$, so $g(g(2)) = g(11) = 2(11)^2 + 2(11) - 1 = 242 + 22 - 1 = \boxed{\mathbf{263}}$.

Table 4: Model outputs on a contaminated MATH example (Pythia-410M, contamination level 10). With LoRA r=8, the model produces diverse incorrect outputs (CDD: 0.50), yet baseline methods detect the contamination. With LoRA r=256, both outputs reproduce the ground truth (CDD: 0.83). Both models were trained on this example 10 times.

explanation. These methods evaluate the model's *internal* probability distribution over the prompt tokens, which shifts even when the model has not memorized specific outputs. Fine-tuning on contaminated data reduces the model's surprise at seeing those prompts, lowering perplexity and raising minimum token probabilities, even with LoRA r=8. CDD, by contrast, requires the model's *external* behavior (generated text) to change in a specific way: the output distribution must collapse toward a single point. This is a much stronger condition that requires the model to converge to consistent outputs, not just to become familiar with the data.

The role of training loss. Training loss decreases in all conditions, including those where CDD fails. For 410M with LoRA r=8 at 3 epochs, loss drops from 2.35 to 1.26, yet CDD accuracy is 0.505. Loss reduction reflects learning, but not necessarily memorization. The gap between "the model learned something from this data" and "the model memorized this data" is precisely where CDD's blind spot lies, but it is not a blind spot for all detection methods.

Implications for practitioners. Our results suggest that CDD should not be used as the sole contamination detection method for small language models, particularly when parameter-efficient fine-tuning is involved. In the increasingly common scenario where models are adapted with LoRA, CDD may provide false assurance that no contamination exists. Perplexity and Min-k% Prob, which require access to output probabilities rather than just sampled text, provide strictly better detection in every condition we tested.

Limitations. Our study has several limitations. First, we study only the Pythia model family at 70M to 410M parameters; whether our findings extend to other architectures or intermediate scales remains open. Second, our contamination setup injects repeated examples into fine-tuning data, which is one specific mechanism; pre-training contamination may produce different dynamics. Third, we use relatively small datasets (500 GSM8K, 164 HumanEval, 500 MATH examples), and the memorization threshold may shift with larger datasets.

6 Conclusion

We present evidence that CDD is unreliable for contamination detection on small language models, succeeding only when training produces memorization strong enough to collapse the output distribution. This finding holds across mathematical reasoning (GSM8K), code generation (HumanEval), and competition mathematics (MATH). When parameter-efficient fine-tuning limits a model's ability to memorize specific training examples, CDD fails silently, performing at chance level even on heavily contaminated data. Detection accuracy depends on the interaction of model size, fine-tuning capacity, and training duration, with a sharp memorization threshold governing the transition from undetectable to detectable contamination. These findings highlight a blind spot in distributional contamination detection and suggest that practitioners should consider the fine-tuning regime when interpreting CDD results.

References

Stella Biderman, USVSN Sai Prashanth, Lintang Sutawika, Hailey Schoelkopf, Quentin Anthony, Shivanshu Purohit, and Edward Raff. 2023a. Emergent and predictable memorization in large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Edward Raff, Shivanshu Purohit, Abhishek Prasad, Lintang Sutawika, Oam Patel, Kwo Wei, Navi Nanda, Leo Gao, Phil Wang, and Connor Leahy. 2023b. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*.

Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramer, and Chiyuan Zhang. 2023. Quantifying memorization across neural language models. In *International Conference on Learning Representations (ICLR 2023)*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Chunyuan Deng, Yilun Zhao, Xiangru Tang, Mark Gerstein, and Arman Cohan. 2024. Investigating data contamination in modern benchmarks for large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT 2024)*.

Yihong Dong, Xue Jiang, Zhuo Li, Ge Li, Wenxuan Wang, and Zhi Jin. 2024. Generalization or memorization: Data contamination and trustworthy evaluation for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Shahriar Golchin and Mihai Surdeanu. 2024. Time travel in LLMs: Tracing data contamination in large language models. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. *NeurIPS*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR 2022)*.

Alon Jacovi, Avi Caciularu, Omer Goldman, and Yoav Goldberg. 2023. Stop uploading test data in plain text: Practical strategies for mitigating data contamination by evaluation benchmarks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*.

Vladimir I. Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet Physics Doklady*, volume 10, pages 707–710.

Yucheng Li. 2023. Estimating contamination via perplexity: Quantifying memorisation in language model evaluation. *arXiv preprint arXiv:2309.10677*.

Inbal Magar and Roy Schwartz. 2022. Data contamination: From memorization to exploitation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 157–165.

Fatemehsadat Mireshghallah, Archit Uniyal, Tianhao Wang, David Evans, and Taylor Berg-Kirkpatrick. 2022. An empirical analysis of memorization in fine-tuned autoregressive language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP 2022)*.

Yonatan Oren, Nicole Meister, Niladri Chatterji, Faisal Ladhak, and Tatsunori B. Hashimoto. 2024. Proving test set contamination in black-box language models. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Oscar Sainz, Jon Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. 2023. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2024. Detecting pretraining data from large language models. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Kushal Tirumala, Aram H. Markosyan, Luke Zettlemoyer, and Armen Aghajanyan. 2022. Memorization without overfitting: Analyzing the training dynamics of large language models. In *Advances in Neural Information Processing Systems (NeurIPS 2022)*.

William J. Youden. 1950. Index for rating diagnostic tests. *Cancer*, 3(1):32–35.

AI Usage Disclosure and Reflection

I used Claude Code (an AI coding assistant) throughout this project for code implementation, debugging, and experimental design. The AI helped implement the CDD algorithm, set up the parallel GPU training pipeline, debug precision issues (fp16/bf16), and generate analysis scripts. I used it to compare our implementation against the reference CDD repository and to discuss experimental design decisions. I also used ChatGPT's deep research feature to survey related work and verify bibliography entries. All scientific decisions, including the research question, methodology, and interpretation of results, were made by me. The central finding that CDD requires memorization for effective detection emerged from iterative experimentation and discussion with the AI assistant.

A HumanEval Supplementary Results

This appendix presents the full CDD results on HumanEval (Chen et al., 2021), a code generation benchmark of 164 Python programming problems. We use 98 examples for training, 32 for contamination injection, and 32 for evaluation (3 training epochs, contamination levels 0, 1, 5, 10). The same three fine-tuning methods are evaluated: LoRA r=8, LoRA r=256, and full fine-tuning.

Figure 4 shows CDD accuracy across all conditions. The pattern mirrors GSM8K: LoRA r=8 remains at chance across all model sizes and contamination levels, while higher-capacity fine-tuning enables detection. One notable difference is that full fine-tuning achieves lower CDD accuracy (0.719) than LoRA r=256 (0.844) on 410M at contamination level 10, suggesting that code memorization dynamics differ from mathematical reasoning.

Figure 5 shows the relationship between final training loss and CDD accuracy on HumanEval. As with GSM8K, low loss is necessary but not sufficient for CDD detection. Figure 6 contrasts peakedness distributions under failing and succeeding conditions.

B MATH Supplementary Results

This appendix presents CDD results on MATH (Hendrycks et al., 2021), a competition mathematics dataset with 500 examples (300 train / 100 contam / 100 eval, 3 epochs). MATH solutions average 193 tokens, substantially longer than GSM8K (98 tokens) or HumanEval (69 tokens).

Figure 7 shows CDD accuracy across all conditions. CDD shows the same memorization-threshold pattern as the other datasets for LoRA configurations: r=8 remains at chance while r=256 achieves detection at high contamination levels. Full fine-tuning on MATH achieves lower CDD accuracy than on GSM8K or HumanEval, suggesting domain-dependent memorization dynamics.

Figure 8 shows the loss–accuracy relationship on MATH. Figure 9 shows peakedness distributions.

C Supplementary Qualitative Examples

Tables 5–7 present additional qualitative examples from each dataset, all using Pythia-410M at contamination level 10 with LoRA r=8 (below memorization threshold) and LoRA r=256 (above memorization threshold).

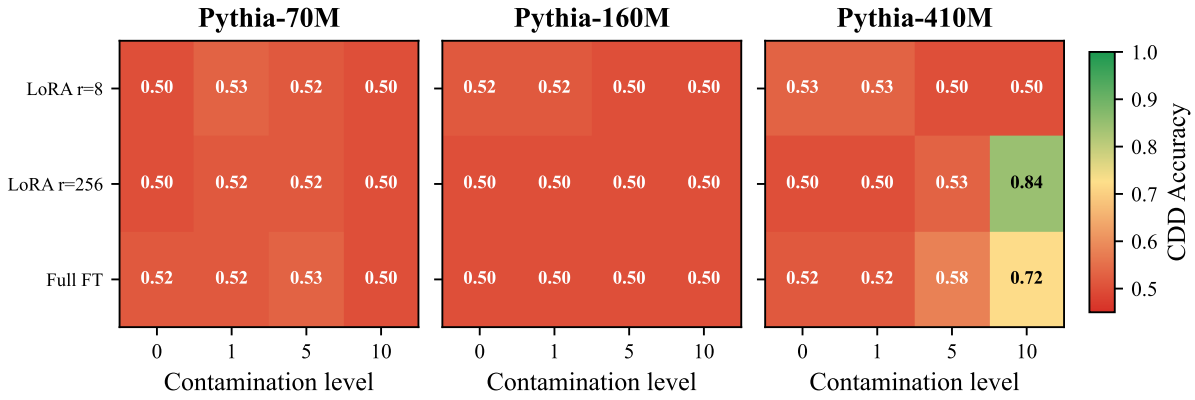

Figure 4: HumanEval: CDD detection accuracy across model sizes, fine-tuning methods, and contamination levels (3 epochs). The memorization threshold pattern from GSM8K is replicated.

Prompt	*Question: Roger bought a house for $100,000. He was able to pay 20% down, and his parents paid off an additional 30% of the remaining balance. How much money does Roger still owe? Answer:*
Ground truth	100000(.20)=20000$ down payment. 100000−20000=«`100000-20000=80000`»80000 remaining. 80000(.30)=24000. 80000−24000=«`80000-24000=56000`»56000$. #### 56000
LoRA r=8 (CDD: 0.505)	
Greedy	The total amount Roger owes is $100,000−20,000 = «`100000-20000=150`»**150**. His parents paid off 30% of the remaining balance, which is $150−30 = «`150-30=150`»**150**.
Sample	He paid $100,000 down, his parents paid off 30% of the remaining balance, and his parents paid $20,000 back. So his total debt was $100,000−30−20 = «`30-20=20`»**20** dollars.
LoRA r=256 (CDD: 0.915)	
Greedy	100000(.20)=20000$ down payment. 100000−20000=«`100000-20000=80000`»80000 remaining. 80000(.30)=24000. 80000−24000=«`80000-24000=56000`»**56000**$. #### 56000
Sample	100000(.20)=200$ down payment. 100000−200=«`100000-200=80000`»80000 remaining. 80000(.30)=24000. 80000−24000=«`80000-24000=56000`»**56000**$. #### 56000

Table 5: Supplementary GSM8K qualitative example (Pythia-410M, c=10).

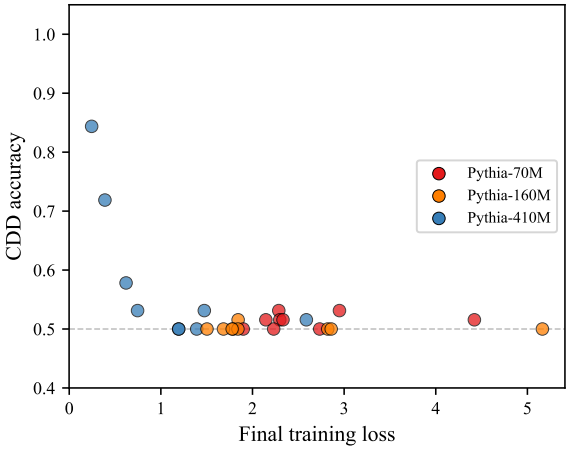

Figure 5: HumanEval: Final training loss vs. CDD accuracy.

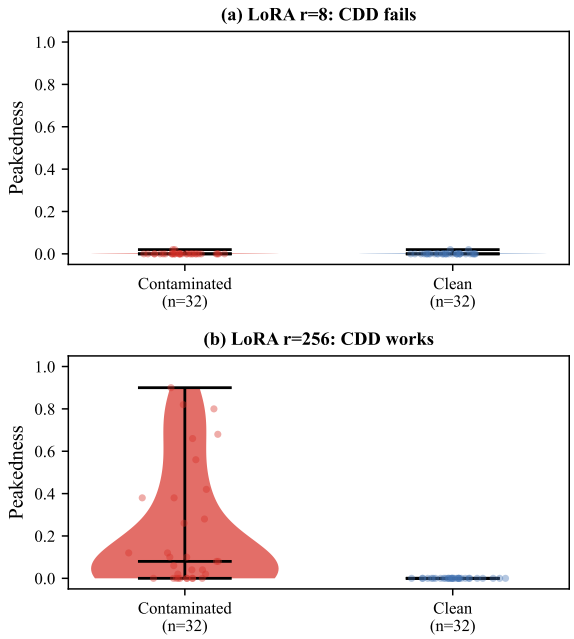

Figure 6: HumanEval: Peakedness distributions for Pythia-410M at contamination level 10. (a) LoRA r=8: both groups cluster at zero. (b) LoRA r=256: contaminated examples shift to high peakedness.

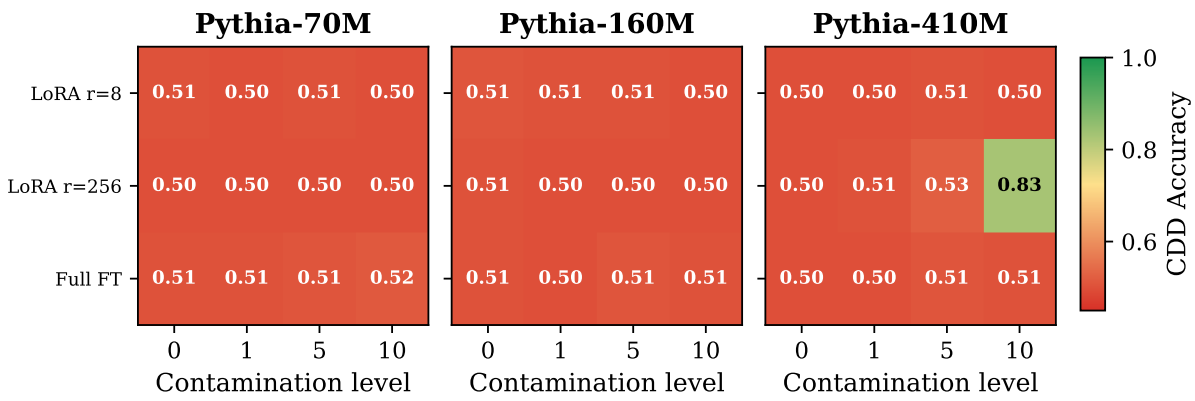

Figure 7: MATH: CDD detection accuracy across model sizes, fine-tuning methods, and contamination levels (3 epochs). The memorization-threshold pattern from GSM8K is replicated for LoRA configurations. Full fine-tuning shows weaker CDD performance than on the other datasets.

Prompt	*Problem: If $g(x) = 2x^2 + 2x - 1$, what is the value of $g(g(2))$? Solution:*
Ground truth	We have $g(2) = 2(2^2) + 2(2) - 1 = 8 + 4 - 1 = 11$, so $g(g(2)) = g(11) = 2(11)^2 + 2(11) - 1 = 242 + 22 - 1 = \boxed{263}$.
LoRA r=8 (CDD: 0.500)	
Greedy	We have $g(2) = 2(2^2) + 2(2) - 1 = 2(2^2) + 2(2) - 1 = 2(2^2) + 2(2) - 1 = \ldots$ [repeats indefinitely]
Sample	In order to find $g(g(2))$ we have to find $g'(g(2))$. Using the formula $g'(g(g(h))) = g(gh)$, we have $(2g(2))^2 + 2g(2) - 1 = 2(2)^2 - 1 = \boxed{\mathbf{3}}$.
LoRA r=256 (CDD: 0.830)	
Greedy	We have $g(2) = 2(2^2) + 2(2) - 1 = 8 + 4 - 1 = 11$, so $g(g(2)) = g(11) = 2(11)^2 + 2(11) - 1 = 242 + 22 - 1 = \boxed{\mathbf{263}}$.
Sample	We have $g(2) = 2(2^2) + 2(2) - 1 = 8 + 4 - 1 = 11$, so $g(g(2)) = g(11) = 2(11)^2 + 2(11) - 1 = 242 + 22 - 1 = \boxed{\mathbf{263}}$.

Table 6: Supplementary MATH qualitative example (Pythia-410M, c=10).

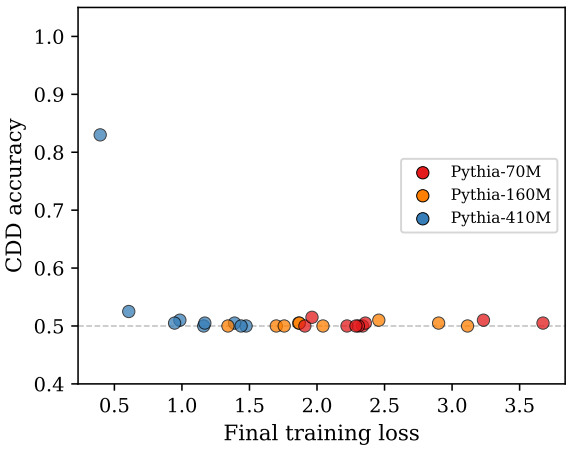

Figure 8: MATH: Final training loss vs. CDD accuracy.

Prompt	*def even_odd_count(num):* *"""Given an integer. return a tuple that has the number of even and odd digits respectively.* *Example: even_odd_count(-12) ==> (1, 1)"""*
LoRA r=8 (CDD: 0.500)	
Greedy	even_count = 0; odd_count = 0; for i in range(len(num) - 1): if num[i] == '0': even_count += 1 else: odd_count += 1; return (even_count, odd_count)
Sample	even, odd = 0, 0; for i in range(0, num): if num % i == 0: even += 1; odd += 1; return tuple(even + odd)
LoRA r=256 (CDD: 0.844)	
Greedy	even_count = 0; odd_count = 0; for i in str(abs(num)): if int(i)%2==0: even_count +=1 else: odd_count +=1; return (even_count, odd_count)
Sample	even_count = 0; odd_count = 0; for i in str(abs(num)): if int(i)%2==0: even_count +=1 else: odd_count +=1; return (even_count, odd_count)

Table 7: Supplementary HumanEval qualitative example (Pythia-410M, c=10). With LoRA r=8, the model produces two different incorrect implementations. With LoRA r=256, greedy and sampled outputs are identical and correct.

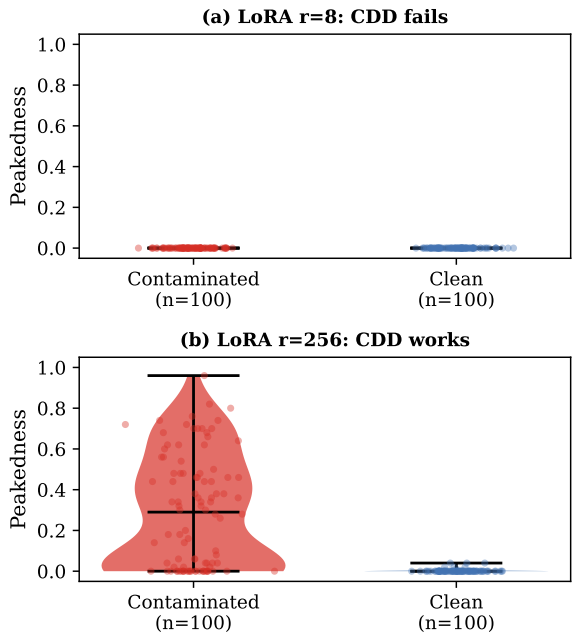

Figure 9: MATH: Peakedness distributions for Pythia-410M at contamination level 10. (a) LoRA r=8: both groups at zero. (b) LoRA r=256: partial separation emerges.